

Path to Least Non-Determinism: A Controlled Method for Turning Agent Skills into Hybrid Workflows

Sai Krishna, Asit Sahoo

Author affiliations, addresses, and corresponding-author details will be added before submission.

Abstract

Language-model agents often repeat simple work because every step of a standard operating procedure is written in natural language. We present Path to Least Non-Determinism (PLaND), an evaluation-driven methodology that keeps judgment in natural language but moves stable work into references, Python, or Bash. The initial system prompt is generated once and frozen after baseline measurement. The model, prompt, harness, data, answers, scorer, seed, and permissions then remain unchanged; only the workflow skill package may evolve. We prepared development, validation, and untouched test partitions and used a prespecified local release gate before opening each test set. On a balanced top-10 single-label LEDGAR subset, the hybrid passed validation and was evaluated once on 1,000 untouched cases. Accuracy changed from 93.5% to 92.7%, a paired difference of -0.8 percentage points with a bootstrap 95% confidence interval from -1.6 to 0.0 points. Tokens fell from 376,088 to 225,573, a 40.0% reduction, and sequential per-case harness latency fell from 0.764 to 0.455 seconds. The hybrid bypassed 411 of 1,000 model calls. On CFPB complaint routing, tokens fell by 41.8%, but accuracy fell from 79% to 72%; the candidate failed both the quality and viability gates. SpamAssassin failed the relative gates, while QS-OCR, SROIE, and an RVL-CDIP mirror failed baseline viability. Their tests remained untouched. These results show the intended behavior of the method: accept measurable savings only when a frozen quality contract is met, and preserve failures as evidence. The current study measures model-mediated work, not run-to-run stochastic variance.

Keywords: agent skills, deterministic workflows, language-model agents, evaluation, token efficiency, workflow optimization, document processing

Introduction

Language-model agents are useful when a task needs interpretation. They can read a document, connect evidence, choose a tool, and handle an exception. They are less useful when they repeatedly interpret the same mechanical instruction. A workflow may ask the model to open a file, normalize a date, count pages, validate a schema, and then make one semantic decision. Most of that work is ordinary computation.

Reusable agent instructions are increasingly packaged as skills. The Agent Skills specification uses a required `SKILL.md` file and may include references, scripts, and assets [1]. DeepAgents can load these skills and support open-ended reasoning and tool use [2]. LangGraph can represent a stable workflow as an explicit graph [3]. These systems provide the building blocks, but they do not decide which parts of an SOP should keep using model judgment.

We began this project with a simple question:

What is the least non-deterministic version of a workflow that still meets its measured quality requirement?

The answer is not "replace all English with code." Natural language is still the right form for ambiguity, context, and judgment. Code is usually the better form for parsing, counting, exact matching, normalization, validation, caching, and fixed calculations. The final workflow can contain both. The code lines bypass unnecessary model reasoning.

This idea is related to work on optimizing language-model programs. DSPy compiles declarative language-model pipelines against a metric [4]. AutoFlow generates and improves natural-language workflows from task descriptions and examples [5]. AFlow searches over code-represented agent workflows [6], and Automated Design of Agentic Systems searches over agent prompts and control structures [7]. Recent systems focus directly on skills. SkillOpt compiles skill packages using execution trajectories and a verifier [8]. SkillRevise updates skills from trace-based failure diagnosis [9]. SkillReducer reduces skill context through compression and progressive disclosure [10]. ACES compares skills in paired evaluations while fixing the task environment [11].

PLaND makes a narrower contribution. It treats the representation of each SOP step as the unit of change and uses a strict mutation boundary. The method can move from an open-ended agent, to a hybrid workflow, and eventually to a mostly deterministic graph when the evidence supports that move. It also records rejected candidates. The purpose is not to maximize determinism. The purpose is to justify each reduction in model-mediated work.

This paper asks three questions:

1. Can a hybrid SOP reduce tokens and latency while staying inside a fixed quality contract?
2. Can a frozen validation gate prevent an apparently efficient but lower-quality candidate from reaching the test set?
3. Which part of a measured saving comes from bypassing model calls rather than merely shortening prompts?

The study does not yet answer whether hybrid SOPs reduce run-to-run output variance. Most cases were run once in each condition. We therefore use "non-determinism" as a design direction and report direct measurements of quality, tokens, calls, latency, and representation.

Material and Methods

PLaND workflow

PLaND uses two skills. `generate-initial-version` reads requirements, approved data sources, and evaluation examples. It creates an initial DeepAgent skeleton and a natural-language workflow SOP. It also generates the system prompt once. `pland-evolver` then studies development traces and proposes bounded changes inside that SOP package.

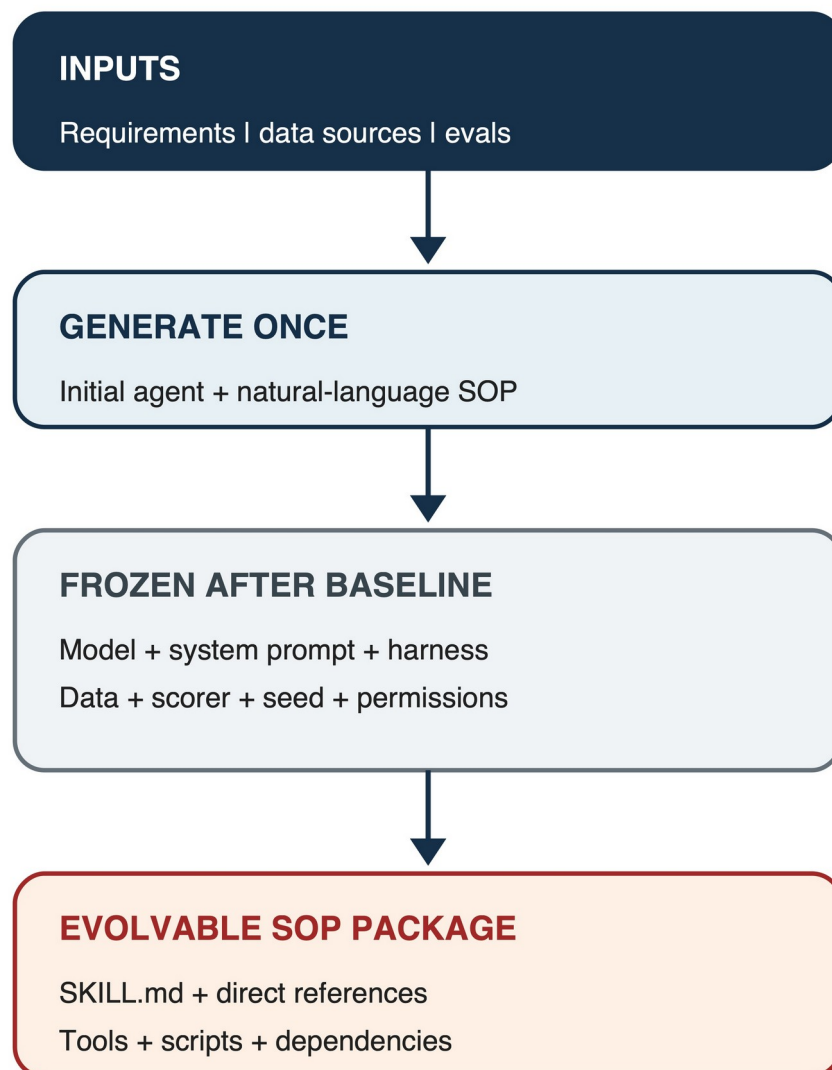


Figure 1. Frozen experimental boundary. The initial agent is generated once. After baseline measurement, only the workflow SOP package may evolve.

Every marked SOP step has one of three forms:

1. **English instruction:** one direct line for work that needs interpretation.
2. **Reference:** a link to another skill file when the instruction is too large for the main SOP.
3. **Command:** a direct Python or Bash invocation for bounded and testable work.

The skill is the unit of capability, the SOP step is the unit of analysis, and a script is the unit of compilation. A hybrid skill may contain all three forms.

For an SOP with N marked steps, we record the model-mediated step share:

$$M = (N_{\text{english}} + N_{\text{reference}}) / N. \quad (1)$$

M is a representation measure. It is not a direct measurement of stochastic behavior.

Evolution path

An unfamiliar task may begin as an open-ended DeepAgents workflow. This is helpful while the agent is still discovering a useful process. Repeated, stable work can then move into code while uncertain decisions remain with the model. If nearly every step becomes stable, the workflow can become a LangGraph graph with intelligence only in the few nodes that still need it.

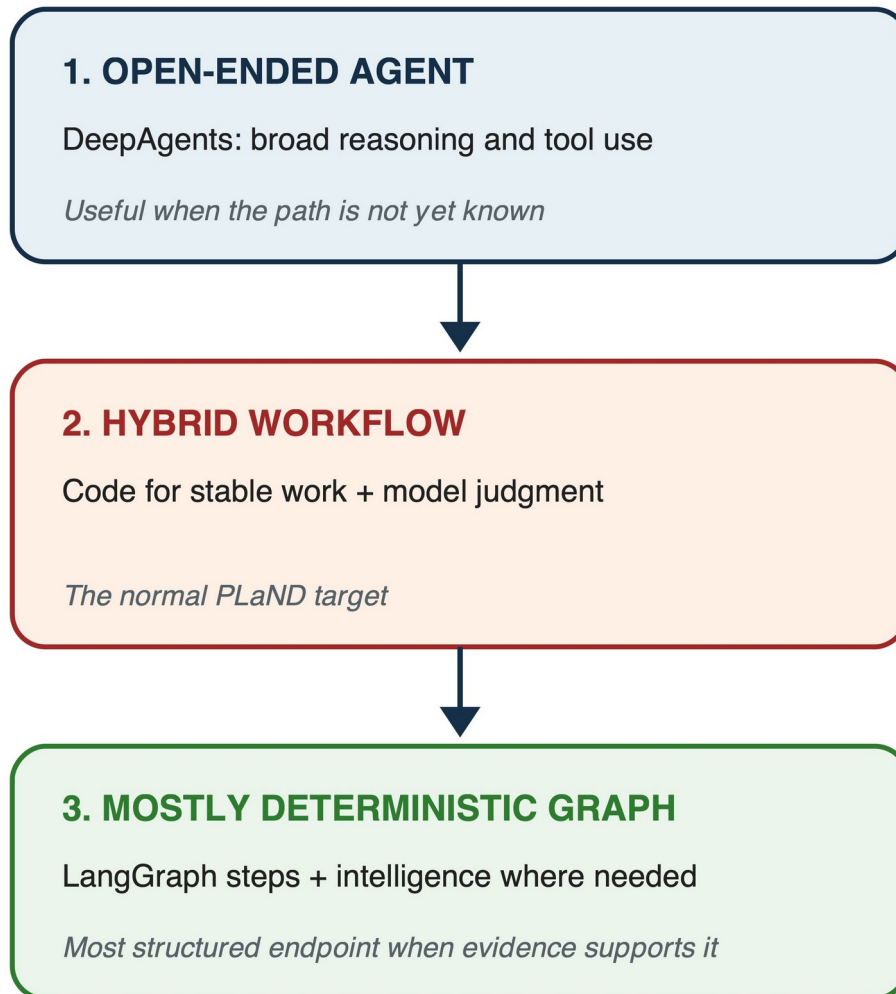


Figure 2. Typical evolution path. The system moves from an open-ended agent to a hybrid workflow and, where evidence permits, to a mostly deterministic graph.

The path is not mandatory. A workflow should stop in the open-ended or hybrid form when further conversion lowers quality.

Frozen controls and allowed changes

After the natural-language baseline is measured, the following items are frozen:

- model name and model digest;
- system prompt;
- agent harness;
- evaluation inputs and expected outputs;
- scorer;
- datasource snapshot;

- seed and model settings; and
- execution permissions.

PLaND may modify only the workflow SOP skill package: `SKILL.md`, its directly referenced instructions, tools and Python or Bash scripts directly invoked by the SOP, and approved dependencies required by those scripts. Candidate checks compare hashes and settings and reject any change outside this boundary.

Generated code must use local or free dependencies unless the workflow explicitly approves another service. Network access may be allowed through a fixed allowlist, including private or VPC services, but the endpoints and permissions must remain unchanged across candidates. The code-generation guidance also asks the evolver to inspect safe caching opportunities and independent steps that could run in parallel. Caching and parallelism are accepted only when observable behavior and experimental isolation remain unchanged.

Bounded evolution loop

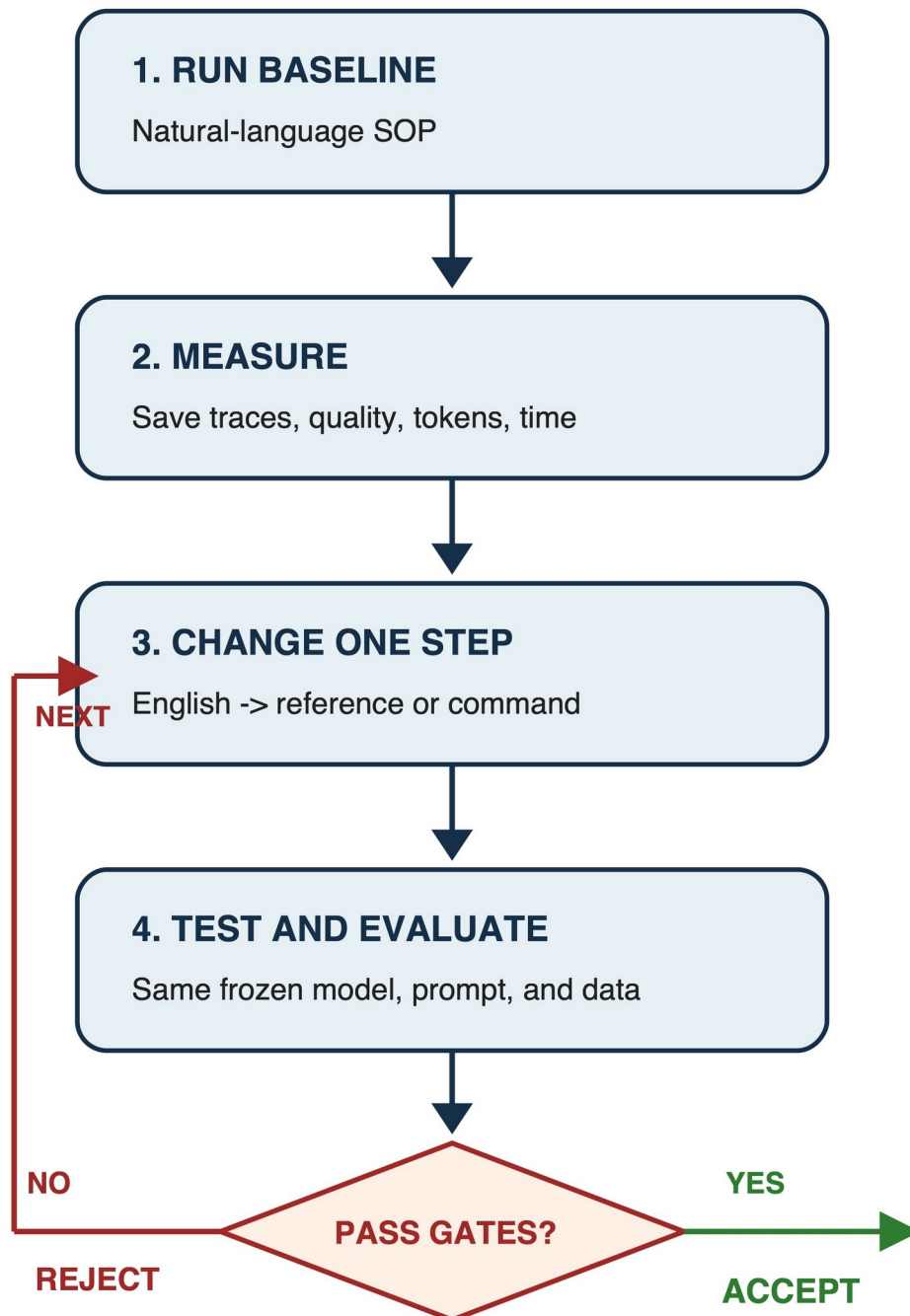


Figure 3. One bounded PLaND iteration. A candidate is accepted only when the frozen invariants, quality gate, and selected expense objective all pass.

The maximum number of candidate iterations is configurable and was set to 10 for this study. Each run saves the exact SOP and its hash, step representations, outputs, errors, traces, model and command calls, tokens, latency, cost estimate, memory, quality metrics, and the acceptance decision.

Dataset preparation

We chose public datasets that resemble enterprise work. LEDGAR contains clauses from public contracts [12]. This study uses a balanced, single-label subset of its ten most frequent eligible provision types; it does not report full-corpus LEDGAR performance. The CFPB complaint database contains consumer narratives and service-routing labels [13]. The SpamAssassin corpus contains public email messages labeled as spam or non-spam [14]. SROIE contains scanned receipts with key information fields [15]. RVL-CDIP and Tobacco3482 contain business document images [16, 17]. LiteParse was included as a local OCR option for image workflows [18].

The confirmatory target was 100 development cases, 100 validation cases, and 1,000 untouched test cases per dataset. Preparation was deterministic. Selection manifests store source identifiers, source and output hashes, seed 20260902, exclusions, and split membership. The audit checks unique identifiers, split overlap, exact content duplicates, missing files, runtime label leakage, source hashes, source boundaries, and overlap with earlier pilot cases.

Table 1. Prepared confirmatory datasets

Workflow	Dev.	Validation	Test	Preparation status
LEDGAR legal-clause classification	100	100	1,000	Passed all preparation checks
CFPB complaint routing	100	100	1,000	Passed all preparation checks
SpamAssassin email classification	100	100	1,000	Passed all preparation checks
QS-OCR/Tobacco3482 document classification	100	100	1,000	Passed all preparation checks
SROIE receipt extraction	100	100	300	Passed; only 300 eligible untouched source-test cases remained
RVL-CDIP document classification mirror	100	100	369	Passed locally; the pinned mirror could not supply 1,000 eligible test cases

The dataset table reports prepared data, not completed model runs. LEDGAR, CFPB, and SpamAssassin have paired validation results; QS-OCR/Tobacco and SROIE have completed natural-language baseline viability results. RVL status is reported below. We did not silently fill source shortfalls by moving development or validation items into the test set.

LEDGAR preserves its upstream train, validation, and test boundaries. SROIE uses its official test partition for test cases; 300 eligible cases remained after exclusions. A strengthened pre-release audit found three test images that duplicated pilot images under different source identifiers. They were removed before any test run, while all 100 validation rows, cases, and images remained byte-identical. The pinned RVL mirror contained only 384 source-test records, and 369 eligible records remained after earlier-pilot and empty-OCR exclusions. A larger RVL test requires a much larger source snapshot and is not represented as completed here.

Evaluation protocol

The text-classification studies used local Ollama 0.33.0 with `qwen3:14b`. The recorded model digest was `bdbd181c33f2ed1b31c972991882db3cf4d192569092138a7d29e973cd9debe8`. Direct paid model cost was therefore recorded as USD 0, although local hardware time and electricity were not free. These studies are a harness-level realization of hybrid SOP routing: the runner first calls the deterministic classifier named by the SOP package and falls back to a direct model request. They do not by themselves demonstrate that a DeepAgent autonomously interpreted and executed the command line. The image-classification runner exercises the generated DeepAgent interface, but its confirmatory natural-language baselines did not reach the viability floor.

For each task, the natural-language and hybrid variants used the same inputs and expected answers. The comparison used an absolute quality floor and a non-inferiority margin. Classification required both accuracies to be at least 0.80. Extraction required field F1 of at least 0.50. The paired bootstrap lower 95% confidence limit for hybrid minus natural-language accuracy had to be at least -0.02. Token reduction had to be at least 5%, and

the lower 95% bootstrap limit for token reduction had to be greater than zero. All frozen invariants had to match exactly.

Only a candidate passing every validation condition could release the untouched test partition. A failed validation stopped the experiment. This rule prevented test labels from becoming another development signal.

For n classification cases, accuracy is:

$$A = (1/n) * \sum_{i=1}^n \text{correct}_i. \quad (2)$$

Let b be the natural-language baseline, h the hybrid, ϵ the allowed quality loss, $q_{\min}$ the minimum viable quality, and δ the minimum worthwhile token reduction. The acceptance rule is:

$$\begin{aligned} &A_h \geq q_{\min}, \\ &\text{lower95}(A_h - A_b) \geq -\epsilon, \\ &\text{token_reduction} \geq \delta, \\ &\text{lower95}(\text{token_reduction}) > 0, \\ &\text{invariants}_h = \text{invariants}_b. \end{aligned} \quad (3)$$

The token reduction is:

$$\text{token_reduction} = (T_b - T_h) / T_b. \quad (4)$$

We used 5,000 paired bootstrap samples with the fixed seed 20260902. We also report Wilson intervals for each accuracy and an exact McNemar test for paired correctness. Latency was measured locally and is descriptive because runs were sequential and were not repeated under a controlled scheduling design.

Reproducibility and data handling

The repository stores preparation scripts, small manifests and hashes, dataset counts, frozen prompts and SOPs, test code, aggregate comparisons, and content-redacted traces that may retain public source identifiers. Large, copyrighted, licensed, or sensitive source data stays outside version control. Reproduction instructions download data from the original source, apply the pinned selection procedure, and verify the saved hashes.

Results

Confirmatory status

Table 2. Confirmatory execution status

Workflow	Validation result	Test release	Interpretation
LEDGAR	Passed all gates	1,000 cases, run once	Confirmatory result
CFPB	Failed quality and viability gates	Not released	Rejected candidate
Email spam	Failed: 90% NL, 86% hybrid; 4.08% token reduction	Not released	Rejected on quality and token gates
QS-OCR	Failed baseline viability: 70% accuracy	Not released	Frozen NL agent was below 80%
SROIE	Failed baseline viability: 0.344 end-to-end field F1	Not released	LiteParse perception baseline was below 0.50
RVL-CDIP	Failed baseline viability: 50% accuracy	Not released	Frozen NL DeepAgent was below 80%

LEDGAR passed validation

On the 100-case LEDGAR validation set, natural-language accuracy was 92% and hybrid accuracy was 93%. Tokens fell from 37,147 to 21,104, a reduction of 43.19%. The paired 95% interval for the accuracy difference was -2 to +5 percentage points, exactly meeting the lower non-inferiority boundary. The lower confidence limit for token reduction was 33.61%, well above zero. Both variants were above the 80% viability floor, so the held-out test was released.

LEDGAR confirmatory test

The 1,000-case LEDGAR test was run once after validation passed. The natural-language SOP classified 935 cases correctly and the hybrid classified 927 correctly. Accuracy was 93.5% and 92.7%, respectively. The paired difference was -0.8 percentage points, with a bootstrap 95% confidence interval from -1.6 to 0.0 points. Natural-language-only correctness occurred in 12 cases, hybrid-only correctness in 4 cases, both were correct in 923 cases, and both were wrong in 61 cases. The exact McNemar p-value was 0.0768.

Tokens fell from 376,088 to 225,573, a reduction of 150,515 tokens or 40.02%. The paired bootstrap 95% confidence interval for the reduction was 37.00% to 43.08%. Model calls fell from 1,000 to 589 because the hybrid routed 411 cases through commands. Mean sequential harness latency fell from 0.764 to 0.455 seconds per case, a 40.4% reduction. Total measured harness latency fell from 764.35 to 455.46 seconds. Peak process memory was similar: 40.39 MB for the natural-language run and 40.30 MB for the hybrid run.

Table 3. Balanced top-10 single-label LEDGAR subset, 1,000-case confirmatory test

Measure	Natural-language SOP	Hybrid SOP	Change
Accuracy	93.5%	92.7%	-0.8 percentage points
Macro F1	0.930	0.922	-0.008
Total tokens	376,088	225,573	-40.02%
Model calls	1,000	589	-411
Command calls	0	411	+411
Mean sequential harness latency	0.764 s	0.455 s	-40.4%
SOP step forms	5 English	3 English + 1 command	Descriptive
Model-mediated step share M	1.00	0.75	-0.25

The result passed the frozen quality, token, viability, and invariant gates. The representation metric changed from five English steps ($M = 1.00$) to three English steps and one command ($M = 0.75$). The total number of steps also changed, so M is descriptive rather than a causal measure. The result should be read as non-inferior within the selected two-point margin, not as proof that both systems are identical.

Where the LEDGAR saving came from

The mechanism can be separated into two parts. On 411 command-routed cases, the natural-language baseline had used 146,366 tokens and the hybrid used no model tokens. On the 589 fallback cases, the natural-language path used 229,722 tokens and the hybrid used 225,573, saving another 4,149 tokens through shorter context. Of the 150,515 total saved tokens, 97.24% came from bypassing model calls and 2.76% came from shortening fallback prompts. This directly supports the proposed mechanism: the code lines bypass unnecessary model reasoning.

Command routing was not perfect. On the same 411 cases, natural-language accuracy was 97.08% and command-routed hybrid accuracy was 96.11%. The overall candidate passed because this loss stayed inside the prespecified margin while producing a large, statistically positive token reduction.

CFPB was rejected before test

The CFPB validation result shows why savings alone are not enough. On 100 validation complaints, natural-language accuracy was 79% and hybrid accuracy was 72%. The paired accuracy difference was -7 percentage points, with a bootstrap 95% confidence interval from -13 to -2 points. The exact McNemar p-value was 0.0391. The baseline also fell just below the 80% absolute viability floor.

Tokens fell from 60,514 to 35,247, a 41.75% reduction. Mean measured latency fell from 1.119 to 0.668 seconds. However, the candidate failed non-inferiority and absolute viability. The 1,000-case CFPB test set was not opened. This is not a failed experimental system; it is the acceptance system working as designed.

Additional validation outcomes

On 100 SpamAssassin validation emails, natural-language accuracy was 90% and hybrid accuracy was 86%. The paired difference was -4 percentage points, with a bootstrap 95% interval from -9 to +1 points. Tokens fell

from 238,077 to 228,359, a 4.08% reduction with a 95% interval from 1.02% to 8.50%. The hybrid bypassed only 3 model calls, and mean latency increased from 4.992 to 5.203 seconds. Although both accuracies exceeded the absolute floor, the candidate failed both the non-inferiority gate and the minimum 5% token-reduction gate. The 1,000-case test remained untouched.

The QS-OCR/Tobacco validation also stopped after the natural-language baseline. On 100 documents, accuracy was 70% and macro F1 was 0.703, below the 80% viability floor. The run used 1,082,556 tokens and averaged 10.626 seconds per case. Because a hybrid could not make both conditions viable without changing the frozen baseline, the hybrid validation condition was not run and all 1,000 test documents remained untouched.

The SROIE end-to-end validation stopped after the natural-language baseline. Across 100 receipts, LiteParse plus the frozen agent achieved field F1 of 0.344, below the 0.50 viability floor. OCR word error rate was 0.676, mean latency was 2.876 seconds, and the run used 49,768 tokens. Because the baseline perception-and-extraction pipeline was not viable, neither the hybrid validation condition nor the corrected 300-case test was run. The earlier 20-case frozen-OCR diagnostic remains useful for separating extraction behavior from OCR quality, but it is not a substitute for this end-to-end gate.

The constrained RVL-CDIP mirror also stopped after the natural-language baseline. On 100 validation documents, the frozen DeepAgent achieved 50% accuracy and 0.472 macro F1, with four invalid-label outputs. It used 1,072,199 tokens and averaged 9.925 seconds per case. Because this was below the 80% viability floor, the hybrid condition was not run and all 369 eligible test documents remained untouched.

Expense at enterprise scale

A per-item saving can look small in a local experiment. Enterprise systems may process millions of documents, emails, or workflow steps. If a measured per-item token saving d remains stable over N comparable items, the illustrative total is:

$$T_{\text{saved}} = N * d. \quad (5)$$

LEDGAR saved an average of 150.515 tokens per item. At one million comparable clauses, that would be about 150.5 million tokens. If a paid model charges p dollars per million tokens, the illustrative direct saving is:

$$C_{\text{saved}} = (T_{\text{saved}} / 1,000,000) * p. \quad (6)$$

This is a scale illustration, not a production forecast. Actual savings depend on input length, output length, batching, caching, traffic mix, hardware, model choice, and price.

Discussion

Main finding

The clearest finding is not that hybrid is always better. It is that a controlled method can tell us when hybrid is acceptable. LEDGAR met a fixed quality contract while using substantially fewer tokens and model calls. CFPB used fewer tokens but lost too much accuracy, so it was rejected. Both outcomes are useful.

The LEDGAR decomposition also distinguishes two ideas that are often mixed together. Short prompts save some tokens. Bypassing a model call can save many more. In this experiment, more than 97% of saved tokens came from the second mechanism. That makes step representation, not just wording, a meaningful optimization target.

From open-ended agents to graphs

DeepAgents is useful when the workflow is not fully known. A hybrid SOP is useful when some steps have become stable but others still need judgment. LangGraph is a natural endpoint when a reliable pattern and a series of tested steps are known. At that endpoint, most nodes can be deterministic and intelligence can remain only in the few nodes where meaning or exceptions matter.

This progression should be evidence-driven. A rule with poor coverage needs an abstention path back to the model. A weak baseline should be improved before expense optimization. An OCR-dominated task needs a viable perception layer before the downstream SOP can be compared fairly. The goal is the least non-deterministic workflow that passes, not the least expensive workflow at any quality.

Relation to prior work

AutoFlow helps explain how an initial set of steps can be generated: a task description and examples give the model enough context to draft a workflow, and evaluation feedback helps revise it [5]. PLaND can use that approach for the first SOP. Its later question is different: should a stable step remain in natural language, move into a focused reference, or become a command?

SkillOpt is the closest related direction because it treats a skill as editable external state for a frozen agent and accepts bounded changes against validation evidence [8]. PLaND adds an explicit natural-language-to-reference-to-command representation path, freezes a small experimental boundary, and requires an untouched test release gate. SkillRevise and SkillReducer are complementary: better failure diagnosis and smaller skill context can be used inside the allowed mutation package [9, 10]. ACES motivates paired evaluation but asks a different comparison question: PLaND compares two representations of the same skill rather than skill versus no skill [11].

Cost, resources, and carbon

Local Ollama avoided paid API charges, but local inference still used time, hardware, and electricity. We measured tokens, latency, model calls, and process memory. We did not measure electrical power, so we do not claim a measured carbon reduction.

A future study can measure average device power P in watts over runtime t in seconds:

$$\text{energy_kWh} = (P * t) / 3,600,000. \quad (7)$$

If the electricity mix has carbon intensity g grams of carbon-dioxide equivalent per kilowatt-hour, then:

$$\text{CO2e_grams} = \text{energy_kWh} * g. \quad (8)$$

Inference energy changes with hardware, model, batching, utilization, and output length [19, 20]. Token and latency reductions are useful operating measures, but they are not direct carbon measurements.

Limitations

This study has several limits. First, the confirmatory claim currently rests on one 1,000-case LEDGAR test and one local model. Second, most cases were run once, so the study does not directly measure behavioral variance. Third, latency can move with warm-up, caching, thermal state, and other work on the same computer. Fourth, the selected 2-point non-inferiority margin is an engineering decision, not a universal standard. Fifth, author-designed candidate rules may not transfer to another domain. Sixth, dataset labels may contain errors. Seventh, a pinned RVL mirror could not provide the full requested 1,000-case test, and SROIE had only 300 eligible untouched test cases. Eighth, direct paid model cost was zero in this local setup, so tokens and latency are the main expense measures. Ninth, the positive text-classification result uses harness-level routing rather than autonomous DeepAgent command interpretation; an end-to-end DeepAgent comparison remains future work. Tenth, some reproducibility fields absent from the original text-run payloads were checked in a clearly labeled post-run artifact audit rather than enforced as first-class runtime fields. Eleventh, the current experiments do not compare against a shorter-natural-language SOP, a fully deterministic workflow, or a no-skill baseline. Finally, a maximum of 10 iterations cannot prove that the globally best workflow was found.

Future work should repeat each condition K times with paired seeds and runtime controls. For categorical output, case-level disagreement can be measured as:

$$D_i = 1 - \max_y \text{count}_i(y) / K. \quad (9)$$

The repeated study should report mean disagreement, pairwise output agreement, tool-sequence agreement, all-runs-correct rate, and token and latency variance. It should also include ablations for shorter natural language, hybrid, deterministic-only where feasible, and caching or parallelism on and off.

Conclusion

PLaND is a controlled methodology for moving suitable agent work from natural language into tested code. It generates an initial agent and system prompt once, measures a natural-language baseline, freezes everything outside the SOP package, and keeps a candidate only when quality remains acceptable and a chosen expense improves.

The 1,000-case balanced top-10 single-label LEDGAR subset provides the strongest evidence so far. The hybrid stayed inside a two-point quality margin, reduced tokens by 40.0%, reduced measured sequential harness latency by 40.4%, and replaced 411 model calls with command calls. This does not estimate performance on the complete multi-label LEDGAR benchmark or on production class prevalence. The CFPB experiment provides an equally important counterexample: a 41.8% token reduction was rejected because accuracy fell too far and the baseline missed the viability floor.

The practical lesson is simple. Keep model judgment where it adds value. Move stable operations into ordinary code. Freeze the comparison. Measure the whole workflow. At enterprise volume, small per-item savings can become large, but a saving counts only after the quality gate passes. Across six prepared studies, only LEDGAR reached a confirmatory test; the five stopped studies are evidence that the gate prevents weak baselines or unsafe optimizations from becoming favorable test claims.

Acknowledgements

The authors thank the people who reviewed the workflow design and dataset choices. Specific acknowledgements and funding information will be added only with permission.

Disclosure and Conflict of Interest

The authors will provide the final journal-required disclosure and conflict-of-interest statement before submission.

Data and Code Availability

The project repository stores the methodology, skill definitions, dataset-preparation scripts, proof manifests, tests, and aggregate experimental results. A public repository URL and archival identifier will be added before submission. Large or restricted source data is not redistributed; its source, revision where available, selection procedure, and hashes are recorded so an authorized user can reproduce the prepared subset.

References

- [1] Agent Skills. (2026). *Agent Skills specification*. <https://agentskills.io/specification>
- [2] LangChain. (2026). *DeepAgents: Skills*. <https://docs.langchain.com/oss/python/deepagents/skills>
- [3] LangChain. (2026). *LangGraph overview*. <https://docs.langchain.com/oss/python/langgraph/overview>
- [4] Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., & Potts, C. (2023). DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv*. <https://arxiv.org/abs/2310.03714>
- [5] Li, Z., Xu, S., Mei, K., Hua, W., Rama, B., Raheja, O., Wang, H., Zhu, H., & Zhang, Y. (2024). AutoFlow: Automated workflow generation for large language model agents. *arXiv*. <https://arxiv.org/abs/2407.12821>
- [6] Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., Zheng, B., Liu, B., Luo, Y., & Wu, C. (2024). AFlow: Automating agentic workflow generation. *arXiv*. <https://arxiv.org/abs/2410.10762>
- [7] Hu, S., Lu, C., & Clune, J. (2024). Automated design of agentic systems. *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2408.08435>
- [8] Yang, Y., Gong, Z., Huang, W., Yang, Q., Zhou, Z., Huang, Z., Li, Y., Gao, X., Dai, Q., Liu, B., Qiu, K., Yang, Y., Chen, D., Yang, X., & Luo, C. (2026). SkillOpt: Executive strategy for self-evolving agent skills. *arXiv*. <https://arxiv.org/abs/2605.23904>
- [9] Liu, Y., Su, Z., Xie, L., Zhang, Y., Zong, Q., Guo, J., Xie, Z., Ji, Y., Yim, Y., Luo, H., Ren, X., Chenyu, R., Li, H., & Song, Y. (2026). SkillRevise: Improving LLM-authored agent skills via trace-conditioned skill revision. *arXiv*. <https://arxiv.org/abs/2606.01139>
- [10] Gao, Y., Li, Z., Yuan, Y., Ji, Z., Ma, P., & Wang, S. (2026). SkillReducer: Optimizing LLM agent skills for token efficiency. *arXiv*. <https://arxiv.org/abs/2603.29919>
- [11] Kevin, C., Raghavan, N., Puget, J.-F., Malani, R., Puvvadi, M., Abramovitch, M., Gupta, M., Akkiraju, R., Prabhu, S., Dangi, Y., Luo, W., & Lee, S. H. (2026). Evaluating skills, not just agents: Agentic continuous evaluation of skills. *arXiv*. <https://arxiv.org/abs/2608.20614>
- [12] Tuggenor, D., von Däniken, P., Peetz, T., & Cieliebak, M. (2020). LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts. *Proceedings of LREC 2020*, 1235-1241. <https://aclanthology.org/2020.lrec-1.155/>
- [13] Consumer Financial Protection Bureau. (2026). *Consumer Complaint Database*. <https://www.consumerfinance.gov/data-research/consumer-complaints/>
- [14] Apache SpamAssassin. (2006). *Public corpus*. <https://spamassassin.apache.org/old/publiccorpus/>
- [15] Huang, Z., Chen, K., He, J., Bai, X., Karatzas, D., Lu, S., & Jawahar, C. V. (2021). ICDAR2019 competition on scanned receipt OCR and information extraction. *arXiv*. <https://arxiv.org/abs/2103.10213>
- [16] Harley, A. W., Ufkes, A., & Derpanis, K. G. (2015). Evaluation of deep convolutional nets for document image classification and retrieval. *Proceedings of ICDAR 2015*. <https://arxiv.org/abs/1502.07058>
- [17] Lim, G., Larson, S., & Leach, K. (2024). Label errors in the Tobacco3482 dataset. *arXiv*. <https://arxiv.org/abs/2412.13140>
- [18] LlamaIndex. (2026). *LiteParse: Open-source document parsing*. <https://github.com/run-llama/liteparse>
- [19] Li, B., Jiang, Y., Gadepally, V., & Tiwari, D. (2024). Sprout: Green generative AI with carbon-efficient LLM inference. *Proceedings of EMNLP 2024*, 21799-21813. <https://aclanthology.org/2024.emnlp-main.1215/>
- [20] Fernandez, J., Na, C., Tiwari, V., Bisk, Y., Luccioni, S., & Strubell, E. (2025). Energy considerations of large language model inference and efficiency optimizations. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 1, 32556-32569. <https://doi.org/10.18653/v1/2025.acl-long.1563>